Ecole Nationale des Sciences Appliquées de Fès
Université Sidi Mohamed Ben Abdellah

TP 1

Automatisation de Réseaux avec Python

Module Automatisation des Réseaux

Prérequis : Bases Python • Réseaux TCP/IP • SSH

Outils : GNS3 2.x • Python 3.x • Netmiko • Paramiko

BELHAJ KHAOULA

N° 9

GSCSI - G1

15 Avril 2026

Introduction

Ce travail pratique porte sur l'automatisation des équipements réseau Cisco à l'aide du langage Python et des bibliothèques SSH spécialisées Paramiko et Netmiko. L'environnement de simulation GNS3 est utilisé pour émuler des routeurs Cisco IOS sans matériel physique.

L'objectif principal est de passer d'une gestion manuelle des équipements réseau (ligne de commande CLI) à une gestion automatisée et scriptée, capable d'auditer, surveiller, configurer et générer des rapports sur l'infrastructure réseau.

Outil	Version	Rôle
GNS3	2.2.47	Simulation réseau — routeurs Cisco virtuels
Python	3.8.10	Langage de scripting pour l'automatisation
Paramiko	2.6.0	Bibliothèque SSH bas niveau
Netmiko	4.4.0	Bibliothèque SSH spécialisée réseau
Ubuntu	20.04	Système d'exploitation hôte

Partie 1 — Préparation de l'environnement

La mise en place de l'environnement de travail constitue le fondement du TP. Elle comprend la vérification des outils existants, l'installation des bibliothèques manquantes et la configuration du réseau entre Ubuntu et GNS3.

1.1 Vérification de l'environnement initial

Les commandes de vérification ont révélé que GNS3 et Python 3 étaient présents mais que pip3 était absent du système.

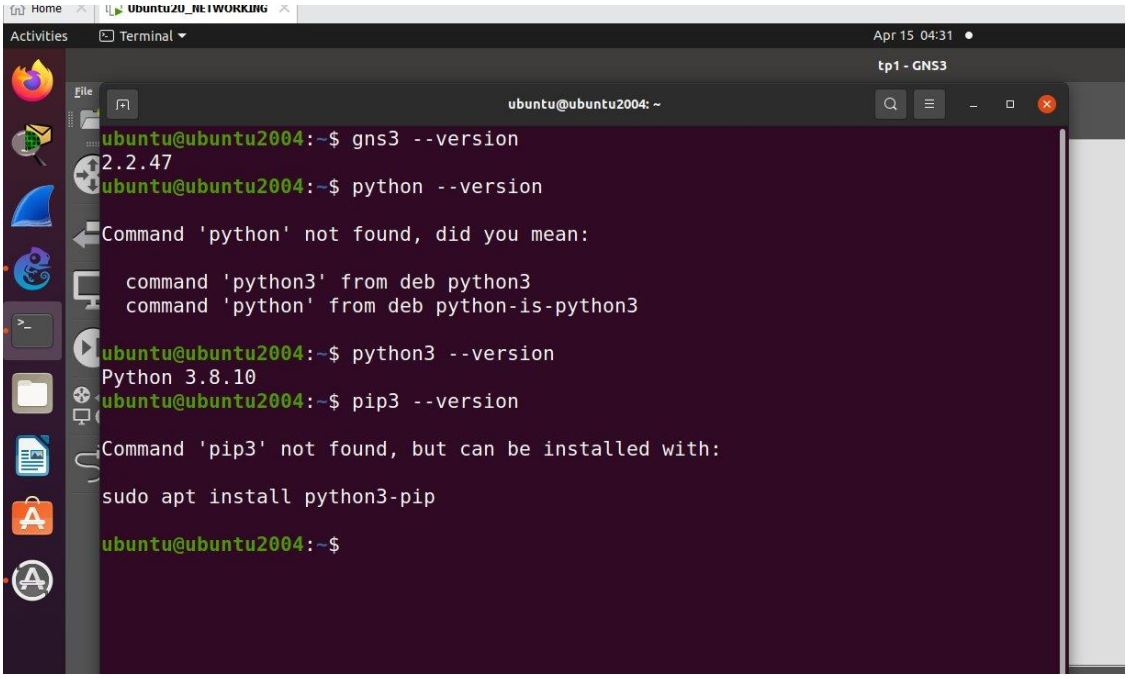


Figure 1 — État initial : GNS3 2.2.47 ✓ Python 3.8.10 ✓ pip3 ✗

1.2 Résolution du problème réseau VMware

Un problème de connectivité Internet a été rencontré lors de l'installation de pip3. La cause : la VM Ubuntu était configurée en mode Bridged dans VMware, sans passerelle réseau disponible. La solution a consisté à passer en mode NAT.

> ⚠ Problème : sudo apt install python3-pip échoue — No address associated with hostname
> ⚠ Cause : mode réseau Bridged sans accès Internet dans VMware
> ⚠ Solution : changer le Network Adapter en mode NAT dans VMware

1.3 Installation des bibliothèques

Après correction du réseau, l'installation de pip3 et des bibliothèques du TP s'est déroulée sans erreur.

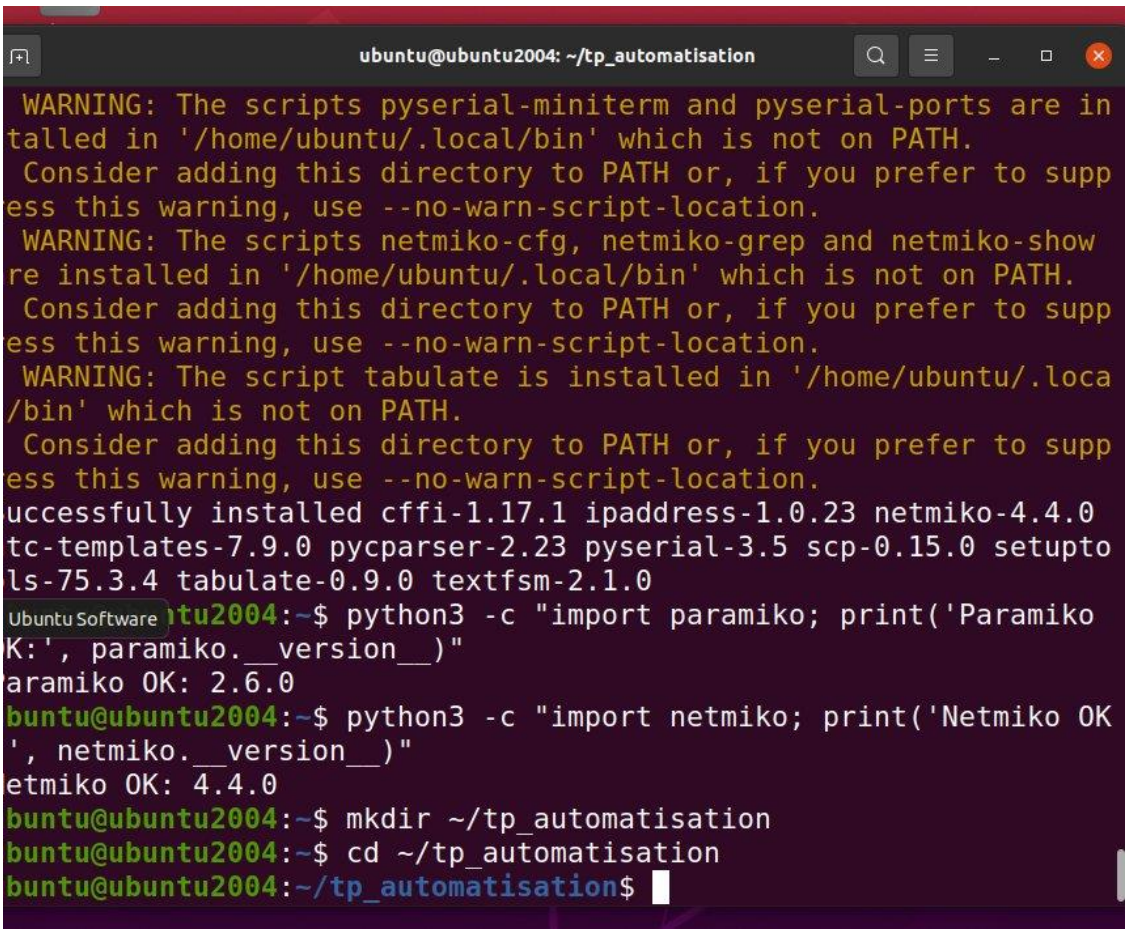


Figure 2 — Installation réussie de Paramiko 2.6.0 et Netmiko 4.4.0

✓ pip3 installé via sudo apt install python3-pip
✓ Paramiko 2.6.0 — bibliothèque SSH bas niveau
✓ Netmiko 4.4.0 — bibliothèque SSH spécialisée réseau
✓ Dossier ~/tp_automatisation créé comme espace de travail

Partie 2 — Topologie 1 : Routeur unique R1

2.1 Description de la topologie

La première topologie est composée d'un seul routeur Cisco c7200 (R1) connecté à un nœud Cloud (Cloud1). Le nœud Cloud fait office de pont entre le réseau virtuel GNS3 et le système Ubuntu hôte, permettant aux scripts Python d'accéder directement au routeur via SSH.

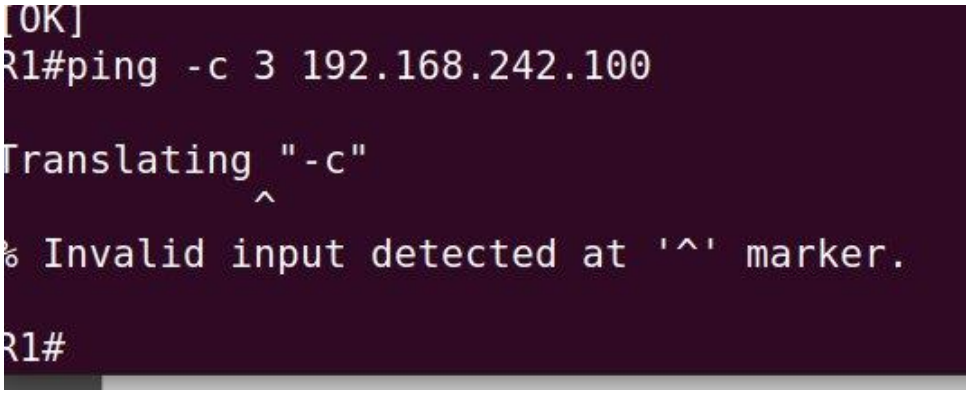


Figure 3 — Topologie 1 dans GNS3 : R1 connecté à Cloud1

2.2 Démarrage de R1 et console IOS

Le routeur R1 (c7200) a été démarré depuis GNS3. L'IOS Cisco 12.4(24)T5 s'est chargé correctement après environ 2 minutes, affichant les messages de démarrage habituels.

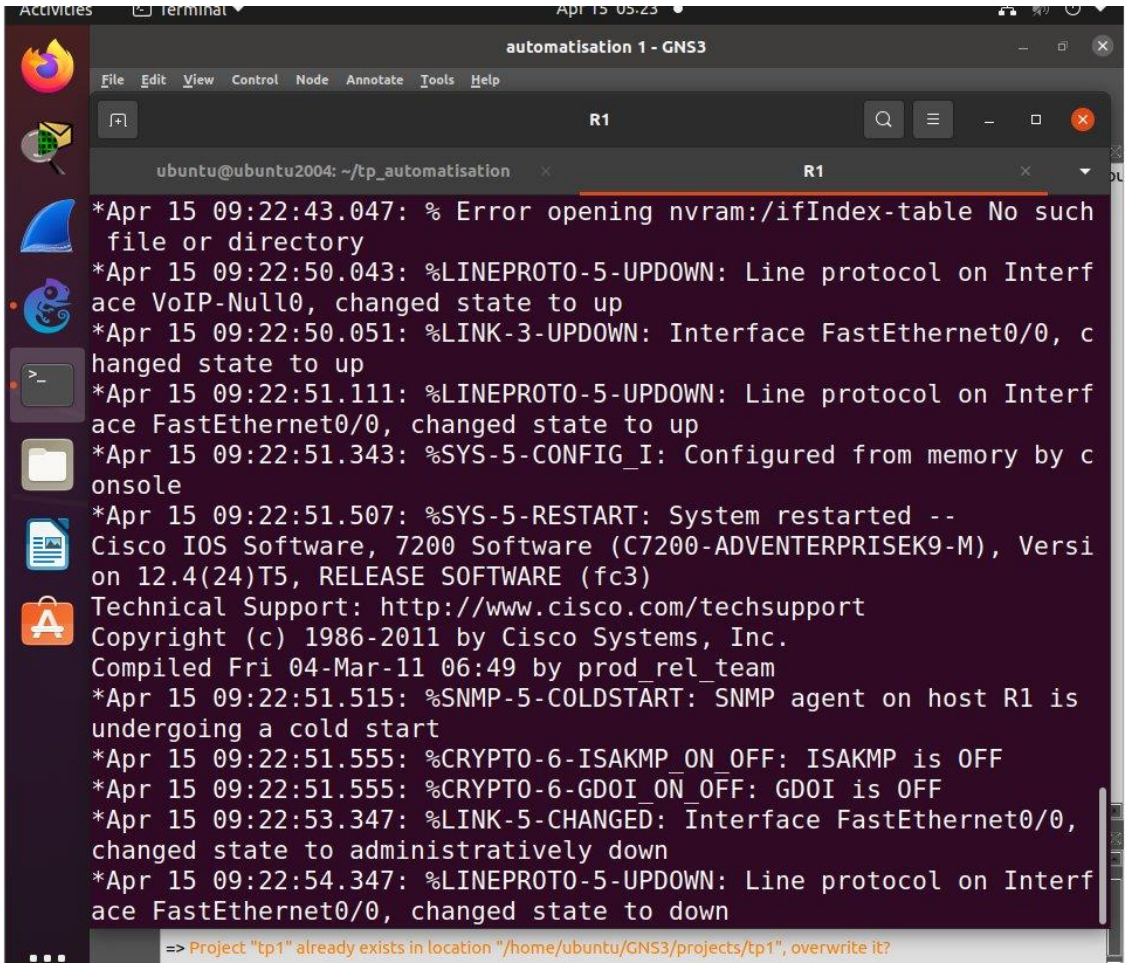


Figure 4 — Console R1 : démarrage IOS Cisco c7200 12.4(24)T5

2.3 Configuration SSH sur R1

La configuration SSH a été réalisée étape par étape dans la console Cisco. Les commandes essentielles sont : configuration de l'interface, domaine IP, génération des clés RSA, création de l'utilisateur et activation SSH v2.

```
R1(config)#ip domain-name lab.local
R1(config)#crypto key generate rsa modulus 1024
The name for the keys will be: R1.lab.local

% The key modulus size is 1024 bits
% Generating 1024 bit RSA keys, keys will be non-exportable...[OK]

R1(config)#
*Apr 15 09:25:23.403: %SSH-5-ENABLED: SSH 1.99 has been enabled
R1(config)#username admin privilege 15 secret cisco123
R1(config)#line vty 0 4
R1(config-line)#transport input ssh
R1(config-line)#login local
R1(config-line)#exit
R1(config)#ip ssh version 2
R1(config)#end
R1#w
*Apr 15 09:27:55.391: %SYS-5-CONFIG_I: Configured from console by
console
R1#write memory
```

Figure 5 — Configuration SSH : génération RSA et activation SSH v2

```
Building configuration...
[OK]
R1#sh ip int brief
Interface                  IP-Address      OK? Method Status
         Protocol
FastEthernet0/0            192.168.148.100 YES manual up
         up
R1#
```

Figure 6 — show ip interface brief : Fa0/0 192.168.122.100 up/up

```
R1#sh ip ssh
SSH Enabled - version 2.0
Authentication timeout: 120 secs; Authentication retries: 3
Minimum expected Diffie Hellman key size : 1024 bits
R1#
```

Figure 7 — Confirmation : SSH Enabled - version 2.0

2.4 Résolution des problèmes de connectivité

Deux problèmes successifs ont été résolus :

1. Sous-réseaux incompatibles : Ubuntu (192.168.242.x) et R1 (192.168.148.x) n'étaient pas dans le même réseau. Solution : adapter l'IP de R1 au réseau Ubuntu.
2. Interface Cloud1 : le Cloud1 utilisait ens33 au lieu de virbr0. Après changement vers virbr0 et adaptation de R1 en 192.168.122.100, la connectivité a été rétablie.
3. Algorithmes SSH : Ubuntu récent et IOS ancien utilisent des chiffrements incompatibles. Solution : fichier ~/.ssh/config avec les algorithmes appropriés.

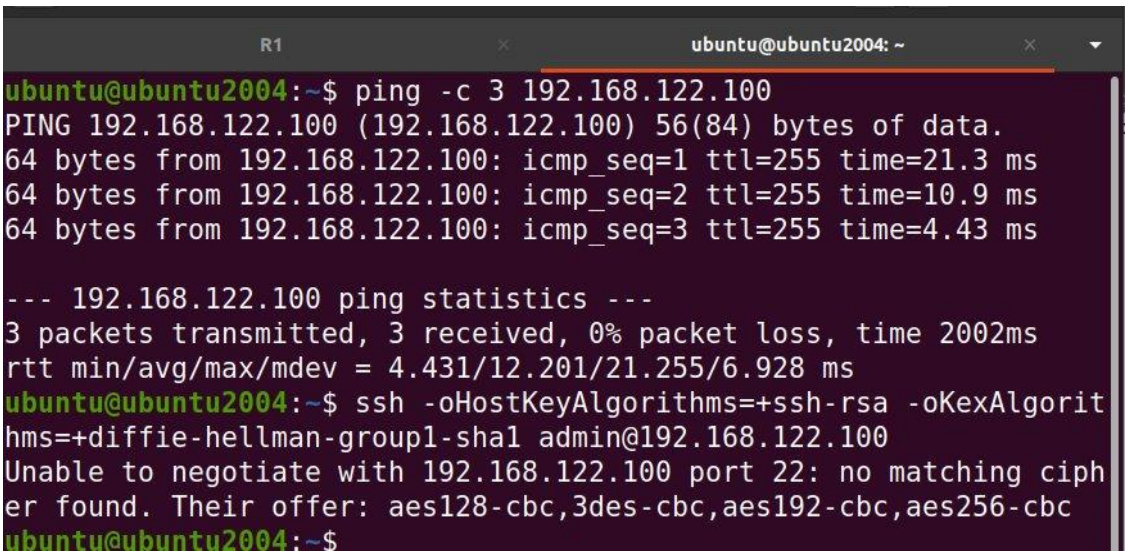


Figure 8 — Ping réussi : Ubuntu → R1 (0% packet loss)

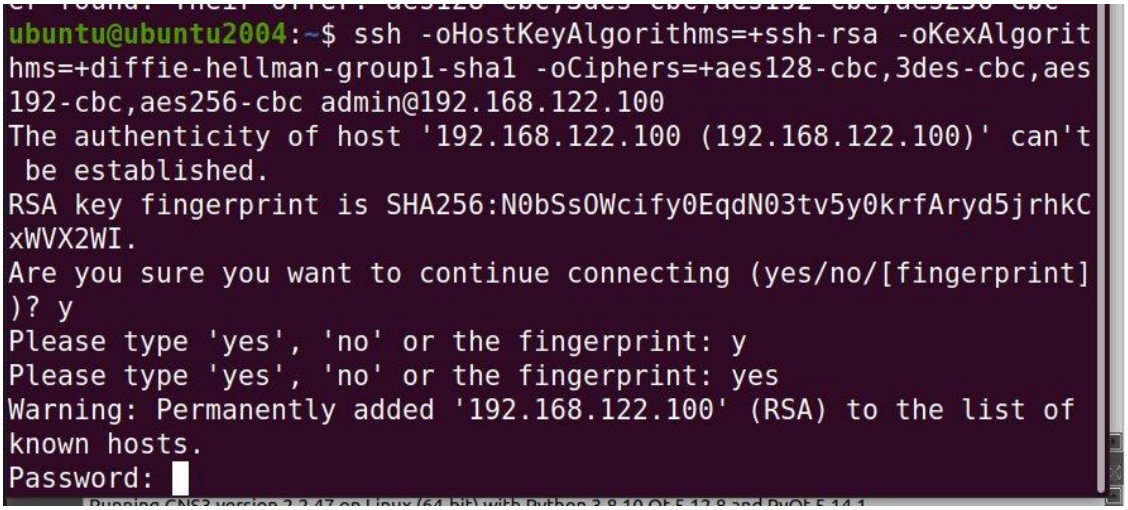


Figure 9 — SSH opérationnel : clé RSA acceptée, prompt R1#

Partie 3 — Scripts Python (Topologie 1)

Trois scripts ont été développés progressivement pour la topologie 1. Chaque script introduit de nouveaux concepts et s'appuie sur les acquis du précédent.

Tâche 1 — Script d'audit complet

Objectif : se connecter à R1 via Netmiko, exécuter 6 commandes de diagnostic et enregistrer les résultats dans un fichier texte horodaté.

Structure du script (5 blocs)
Bloc A : Imports — netmiko.ConnectHandler et datetime
Bloc B : Dictionnaire device — IP, identifiants, type d'équipement
Bloc C : Connexion SSH — ConnectHandler(**device)
Bloc D : Boucle d'audit — send_command() sur chaque commande
Bloc E : Déconnexion — disconnect()

```
ubuntu@ubuntu2004:~/tp_automatisation$ python3 script1_audit.py
=== AUDIT DU ROUTEUR ===

Execution : show version
Execution : show ip interface brief
Execution : show running-config
Execution : show ip route
Execution : show memory statistics
Execution : show processes cpu

Audit termine ! Fichier cree : audit_R1_20260415_062714.txt
ubuntu@ubuntu2004:~/tp_automatisation$
```

Figure 10 — Script 1 : exécution des 6 commandes et création du fichier audit

✓ Connexion SSH Netmiko établie vers 192.168.122.100
✓ 6 commandes exécutées : version, interfaces, config, routes, mémoire, CPU
✓ Fichier audit_R1_20260415_062714.txt créé avec succès

Tâche 2 — Script de monitoring

Objectif : surveiller en continu l'état des interfaces de R1 avec rafraîchissement toutes les 10 secondes. Introduction des concepts de boucle infinie et gestion d'interruption.

Nouveaux concepts Python
while True → boucle infinie de surveillance
try / except → gestion de l'exception KeyboardInterrupt (Ctrl+C)
finally → déconnexion garantie même en cas d'interruption
time.sleep(10) → pause de 10 secondes entre chaque mesure
time.strftime() → affichage de l'heure de chaque mesure

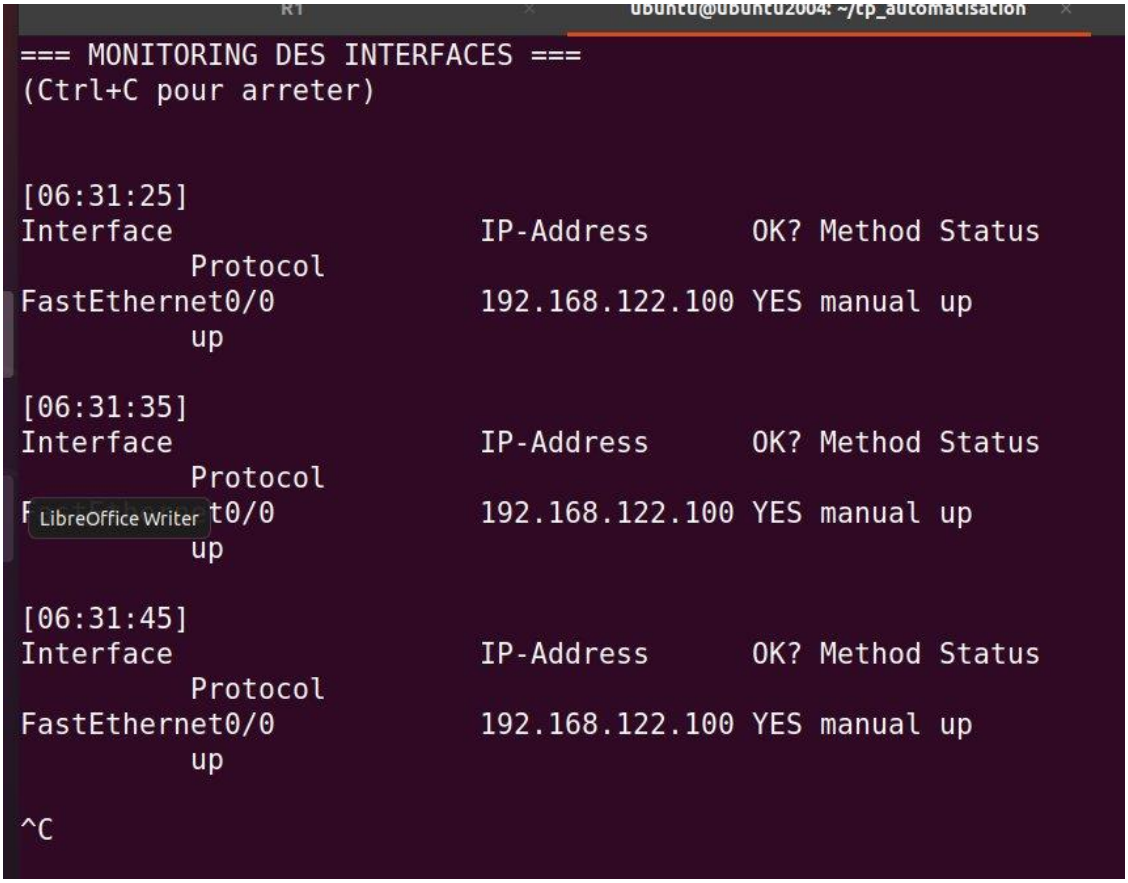


Figure 11 — Monitoring actif : état des interfaces rafraîchi toutes les 10s

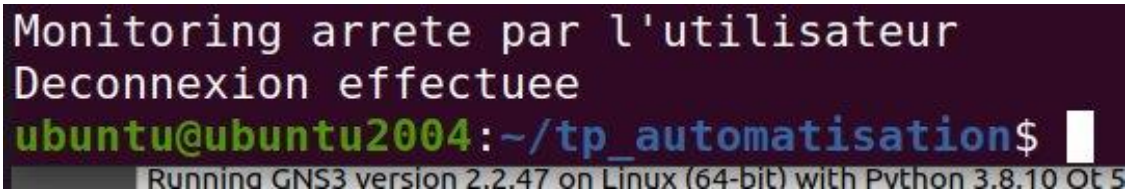


Figure 12 — Arrêt du monitoring via Ctrl+C, déconnexion effectuée

Tâche 3 — Script de configuration automatique

Objectif : configurer les interfaces de R1 automatiquement, vérifier la configuration appliquée et la sauvegarder dans la NVRAM. Introduction de send_config_set() pour l'écriture de configuration.

Différence fondamentale

send_command() → Mode EXEC (lecture seule : show version, show ip int brief...)
send_config_set() → Mode CONFIG (écriture : interface, ip address, description...)
save_config() → Exécute 'write memory' pour persister la configuration

```
ubuntu@ubuntu2004:~/tp_automatisation$ python3 script3_config.py
=== CONFIGURATION AUTOMATIQUE ===

Application de la configuration...
configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface FastEthernet0/0
R1(config-if)#description *** Lien WAN configure par script ***
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#interface FastEthernet0/1
                                  ^
% Invalid input detected at '^' marker.

R1(config)#description *** Interface LAN automatisee ***
             ^
% Invalid input detected at '^' marker.

R1(config)#ip address 10.10.10.1 255.255.255.0
                     ^
% Invalid input detected at '^' marker.
```

Figure 13 — Application automatique de la configuration sur R1

```
R1(config)#no shutdown
              ^
% Invalid input detected at '^' marker.

R1(config)#end
R1#

=== VERIFICATION ===
Interface                  IP-Address      OK? Method Status
          Protocol
FastEthernet0/0            192.168.122.100 YES manual up
          up

=== DESCRIPTIONS DES INTERFACES ===
Interface                      Status         Protocol Description
Fa0/0                          up             up       *** Lien WA
N configure par script ***

Sauvegarde de la configuration...

Configuration terminee et sauvegardee !
ubuntu@ubuntu2004:~/tp_automatisation$
```

Figure 14 — Vérification et sauvegarde : Configuration terminée et sauvegardée

Partie 4 — Topologie 2 : R1, R2 et R3

La deuxième topologie étend l'environnement à trois routeurs Cisco interconnectés. Elle permet de démontrer l'intérêt de l'automatisation : configurer plusieurs équipements simultanément avec un seul script.

4.1 Construction de la topologie dans GNS3

Un nouveau projet GNS3 a été créé avec trois routeurs c7200 (R1, R2, R3), un switch Ethernet L2 (Switch1) et un nœud Cloud1 (virbr0). Tous les routeurs sont reliés au switch qui est lui-même connecté au Cloud.

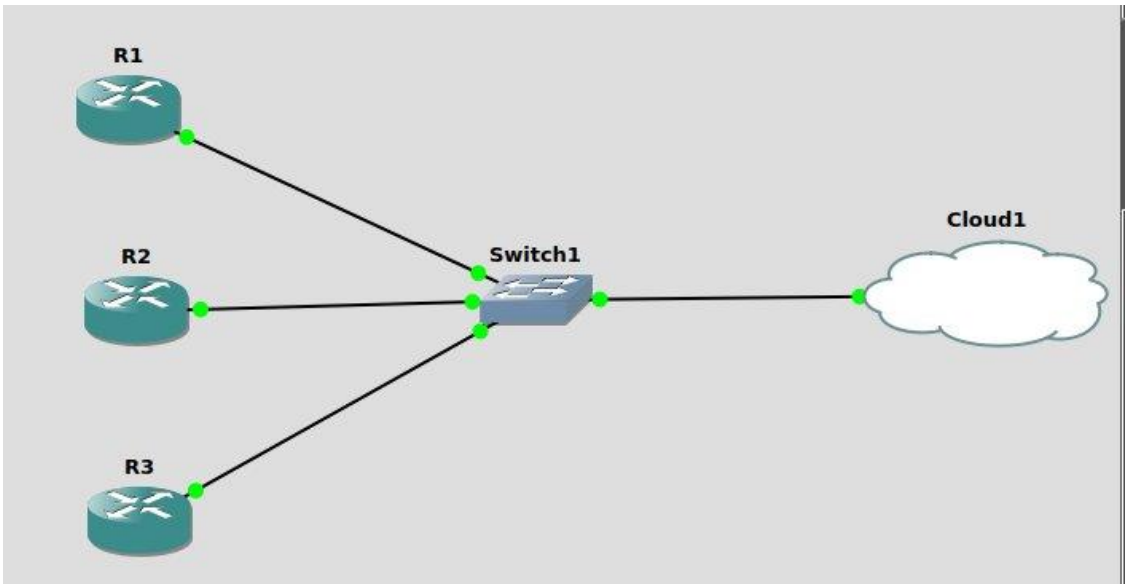


Figure 15 — Topologie 2 : R1, R2, R3 reliés via Switch1 à Cloud1 (virbr0)

4.2 Configuration individuelle des routeurs

Chaque routeur a été configuré avec la même procédure SSH que pour R1 dans la topologie 1, avec des adresses IP différentes : R1=192.168.122.101, R2=192.168.122.102, R3=192.168.122.103.

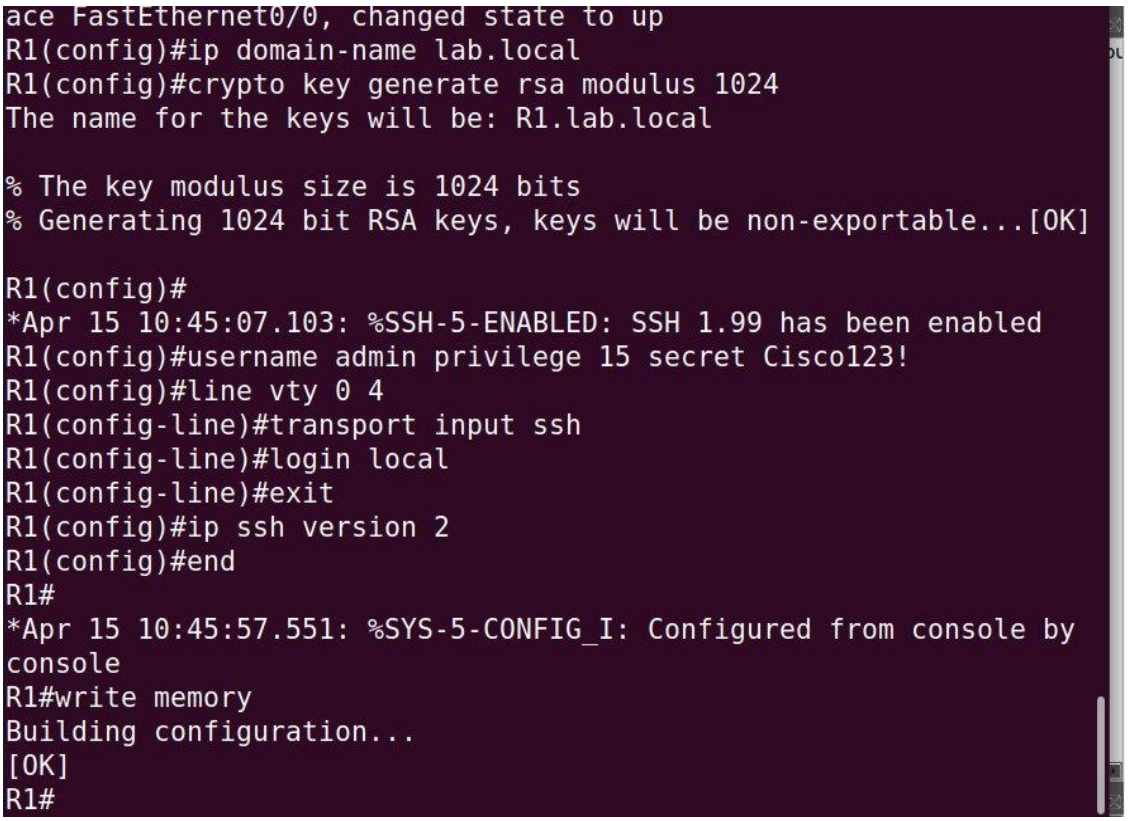


Figure 16 — Configuration de R1 (192.168.122.101)

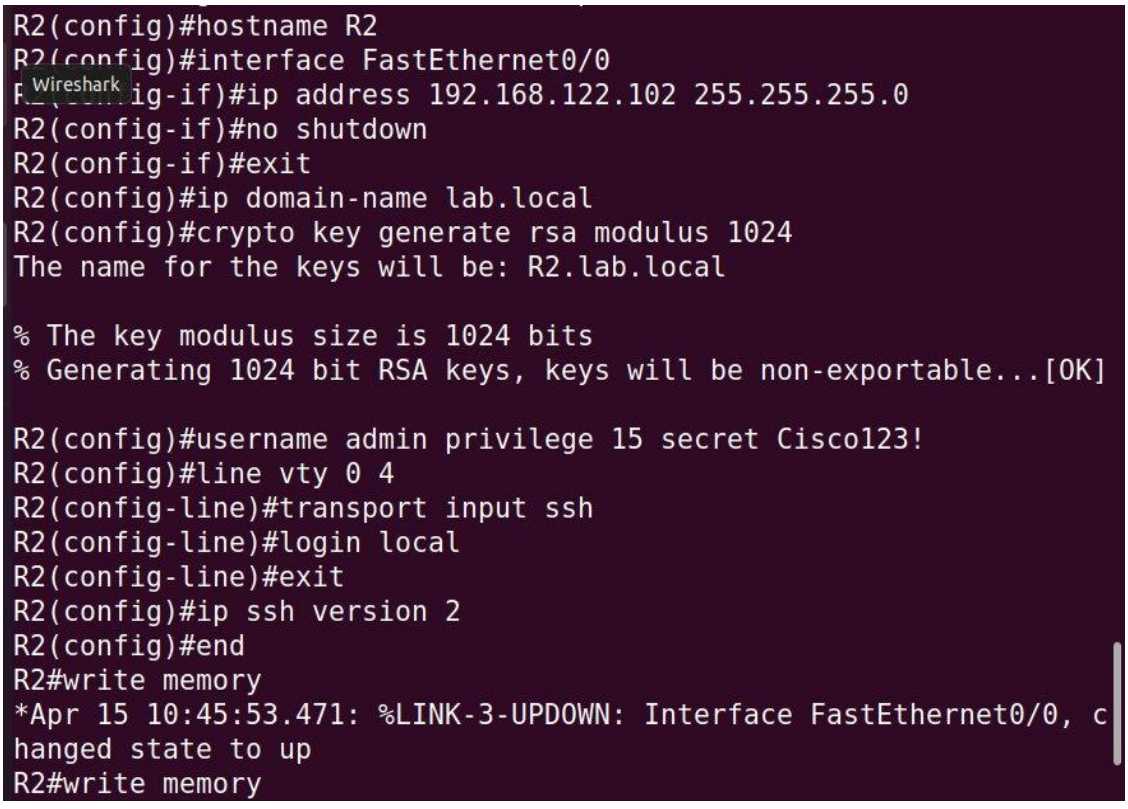


Figure 17 — Configuration de R2 (192.168.122.102)

4.3 Validation de la connectivité

Après configuration de R1, R2 et R3, des pings ont été lancés depuis Ubuntu vers chaque routeur pour valider la connectivité avant d'exécuter les scripts.

```
PING 192.168.122.101 (192.168.122.101) 56(84) bytes of data.
64 bytes from 192.168.122.101: icmp_seq=1 ttl=255 time=54.3 ms
64 bytes from 192.168.122.101: icmp_seq=2 ttl=255 time=2.44 ms

--- 192.168.122.101 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 2.442/28.388/54.335/25.946 ms
ubuntu@ubuntu2004:~$ ping -c 2 192.168.122.102
PING 192.168.122.102 (192.168.122.102) 56(84) bytes of data.
64 bytes from 192.168.122.102: icmp_seq=1 ttl=255 time=23.0 ms
64 bytes from 192.168.122.102: icmp_seq=2 ttl=255 time=9.64 ms

--- 192.168.122.102 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 9.643/16.326/23.010/6.683 ms
ubuntu@ubuntu2004:~$ ping -c 2 192.168.122.103
PING 192.168.122.103 (192.168.122.103) 56(84) bytes of data.
64 bytes from 192.168.122.103: icmp_seq=1 ttl=255 time=22.6 ms
64 bytes from 192.168.122.103: icmp_seq=2 ttl=255 time=10.9 ms

--- 192.168.122.103 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 10.860/16.750/22.641/5.890 ms
```

Figure 18 — Ping vers R1, R2 et R3 : 0% packet loss sur les 3 routeurs

Partie 5 — Scripts Python (Topologie 2)

Tâche 1 — Connexion bas niveau avec Paramiko

Ce script illustre comment Paramiko, la couche SSH sous-jacente à Netmiko, gère une connexion réseau. Il montre les limites de cette approche générique face aux spécificités Cisco (pagination, prompts).

Problème rencontré et solution :

La méthode exec_command() de Paramiko ferme la session après chaque commande sur les vieux IOS Cisco. La solution consiste à utiliser invoke_shell() (session interactive) combiné à la commande terminal length 0 pour désactiver la pagination --More--.

```
ubuntu@ubuntu2004:~/tp_automatisation$ python3 script1_paramiko.py
[*] Connexion SSH vers 192.168.122.101:22...
[+] Connecte avec succes a 192.168.122.101
[>] Envoi de la commande : show version

================================================
Resultat de : show version
================================================
Cisco IOS Software, 7200 Software (C7200-ADVENTERPRISEK9-M), Versi
on 12.4(24)T5, RELEASE SOFTWARE (fc3)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2011 by Cisco Systems, Inc.
Compiled Fri 04-Mar-11 06:49 by prod_rel_team

ROM: ROMMON Emulation Microcode
BOOTLDR: 7200 Software (C7200-ADVENTERPRISEK9-M), Version 12.4(24)
T5, RELEASE SOFTWARE (fc3)

R1 uptime is 12 minutes
System returned to ROM by unknown reload cause - suspect boot_data
[BOOT_COUNT] 0x0, BOOT_COUNT 0, BOOTDATA 19
System image file is "tftp://255.255.255.255/unknown"
```

Figure 19 — Script Paramiko : session interactive, 3 commandes réussies

Comparaison Paramiko vs Netmiko

Paramiko : bibliothèque SSH générique — ne connaît pas les prompts Cisco
Netmiko : s'appuie sur Paramiko + couche intelligente réseau
Netmiko gère automatiquement : pagination, enable, conf t, prompts...
Pour des équipements Cisco, Netmiko est toujours préférable à Paramiko

Tâche 2 — Configuration automatique multi-routeurs

Ce script démontre la puissance de l'automatisation : en quelques secondes, la même configuration est appliquée sur 3 routeurs. Pour chaque routeur, le script active le mode enable, applique les commandes et sauvegarde la configuration.

Pour chaque routeur, l'IP de la loopback est calculée automatiquement depuis le dernier octet de l'IP de gestion (ex: 192.168.122.101 → 10.0.0.101/32).

```
============================================================
 [R3] Mode enable active
 [R3] Prompt : R3#
 [R3] Application de la configuration...
configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R3(config)#interface loopback 0
R3(config-if)#description Loopback-configure-par-Python
R3(config-if)#no shutdown
R3(config-if)#exit
R3(config)#logging buffered 10000
R3(config)#service timestamps log datetime msec
R3(config)#end
R3#
 [R3] IP Loopback configuree : 10.0.0.103/32
 [R3] Configuration sauvegardee

============================================================
 RESUME DE LA CONFIGURATION
============================================================
 [OK] R1 (192.168.122.101) : SUCCES
 [OK] R2 (192.168.122.102) : SUCCES
 [OK] R3 (192.168.122.103) : SUCCES
ubuntu@ubuntu2004:~/tp_automatisation$
```

Figure 20 — Configuration multi-routeurs : R1, R2, R3 — SUCCES

✓ R1 (192.168.122.101) — Loopback 10.0.0.101/32 — SUCCES
✓ R2 (192.168.122.102) — Loopback 10.0.0.102/32 — SUCCES
✓ R3 (192.168.122.103) — Loopback 10.0.0.103/32 — SUCCES

Tâche 3 — Collecte de données et génération de rapports

Le dernier script collecte automatiquement 5 commandes sur chaque routeur (show version, show ip interface brief, show ip route, show processes cpu, show running-config) et génère deux types de rapports complémentaires.

Format	Fichier	Usage
TXT	rapport_reseau_*.txt	Lisible par un humain — audit et documentation
JSON	rapport_reseau_*.json	Exploitable par d'autres outils — traitement automatique

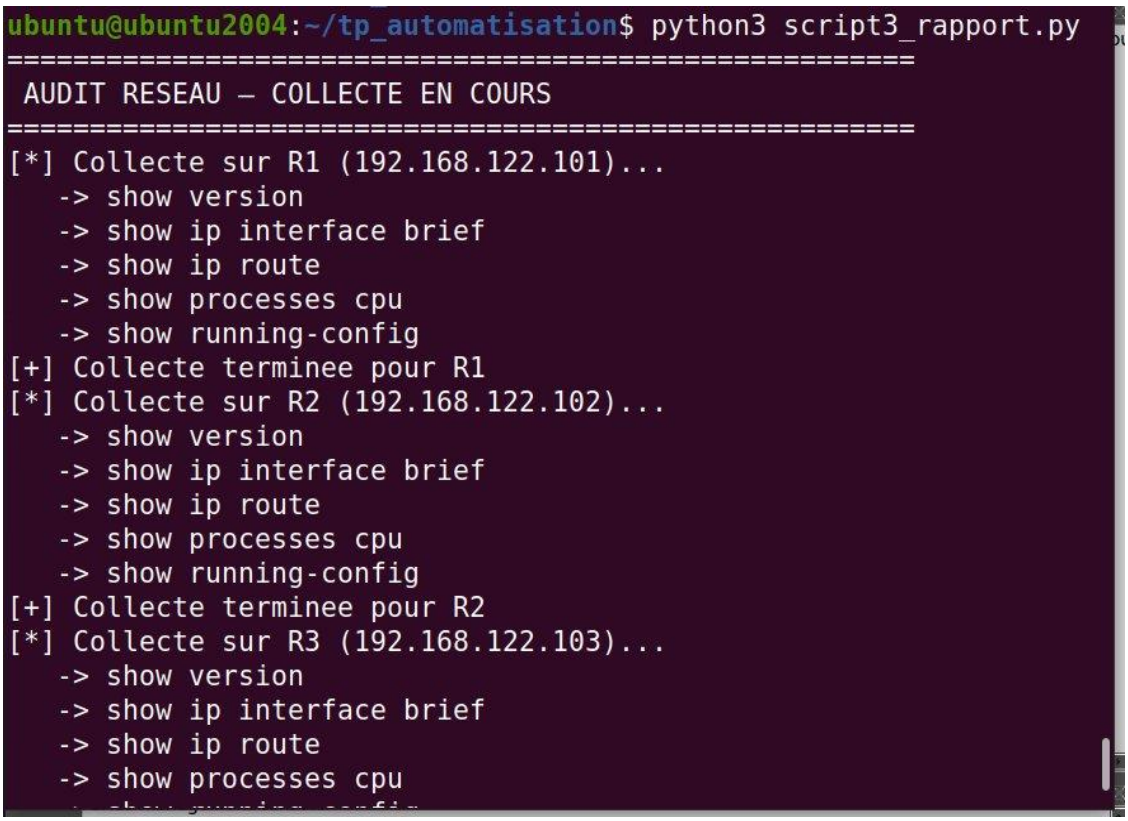


Figure 21 — Collecte en cours : 5 commandes sur chacun des 3 routeurs



Figure 22 — Rapports TXT et JSON générés avec succès

Conclusion et Bilan

Ce TP a démontré comment Python, couplé aux bibliothèques Paramiko et Netmiko, permet de transformer la gestion manuelle des équipements réseau Cisco en un processus automatisé, reproductible et scalable.

Les principales compétences acquises au cours de ce TP sont les suivantes :

4. Mise en place d'un environnement GNS3 avec routeurs Cisco c7200 virtuels sur Ubuntu, incluant la résolution de problèmes réseau VMware et SSH.
5. Configuration SSH complète sur des équipements Cisco IOS (génération RSA, VTY, authentification locale).
6. Compréhension de la relation Paramiko/Netmiko et de leurs cas d'usage respectifs.
7. Développement de scripts d'audit, monitoring et configuration avec Netmiko.
8. Automatisation multi-équipements et génération de rapports structurés (TXT et JSON).

Bilan complet du TP :

Étape	Script/Tâche	Résultat
Partie 1 & 2	**Installation + Topologie 1**	GNS3 2.2.47, Python 3.8.10, Paramiko 2.6.0, Netmiko 4.4.0
Script 1	**Audit routeur**	6 commandes, fichier .txt horodaté généré
Script 2	**Monitoring**	Boucle 10s, arrêt propre Ctrl+C, déconnexion OK
Script 3	**Configuration**	Fa0/0 configurée, description appliquée, write memory
Partie 4	**Topologie 2**	R1(101)+R2(102)+R3(103), ping 0% loss, SSH OK
Script 4	**Paramiko**	invoke_shell, terminal length 0, 3 commandes OK
Script 5	**Netmiko multi**	Loopback IP configurée sur R1, R2, R3 — SUCCES
Script 6	**Rapport JSON+TXT**	Collecte 5 commandes/routeur, 2 fichiers générés

L'ensemble des objectifs du TP a été atteint. Les six scripts ont été développés et exécutés avec succès, validant la maîtrise de l'automatisation réseau Python dans un environnement proche des conditions professionnelles.